WEB PROGRAMCILIĞI III

ÖDEV

Konu: UML Diyagramları (Use Case, ER ve Class Diyagramı Örnekleri)

Hazırlayan: Mehmet Efe Gezici

Öğrenci No: 256001060

Bölüm: Bilgisayar Programcılığı

1. UML Nedir?

UML'nin açılımı Unified Modeling Language, Türkçesi Birleşik Modelleme Dili. Kısaca bir yazılımı kodlamaya başlamadan önce o yazılımı şekillerle çizip planlamamıza yarayan bir dil diyebiliriz. Programlama dili değil, yani UML ile kod yazmıyoruz. Sadece sistemin hangi parçalardan oluşacağını, kimlerin ne yapacağını ve parçaların birbiriyle nasıl bağlantılı olduğunu çiziyoruz.

Bunu bir ev yapmaya benzetebiliriz. Ev yapılmadan önce mimar bir proje çizer, usta da o projeye bakarak evi yapar. Yazılımda da UML diyagramları bu projenin yerini tutuyor. Özellikle birden fazla kişi aynı projede çalışıyorsa herkesin aynı şeyi anlaması için çok işe yarıyor.

UML'yi 1990'larda Grady Booch, James Rumbaugh ve Ivar Jacobson adlı üç kişi geliştirmiş. Bunların her birinin kendi modelleme yöntemi varmış ve bunları birleştirerek UML'yi ortaya çıkarmışlar ("birleşik" kelimesi de buradan geliyor). 1997'de OMG (Object Management Group) adlı kuruluş UML'yi standart olarak kabul etmiş. Şu an kullanılan sürüm UML 2.5.

UML'yi neden kullanıyoruz?

- Ekipteki herkes (müşteri, programcı, test eden kişi) aynı şemaya bakarak konuşabiliyor.
- Hatalar daha kod yazılmadan, tasarım aşamasında fark ediliyor. Bu da sonradan uğraşmaktan kurtarıyor.
- Projeye sonradan katılan biri diyagramlara bakıp sistemi hızlıca anlayabiliyor.
- Hangi programlama dilini kullandığımız fark etmiyor. PHP, C#, Java hepsinde kullanılabiliyor.
- StarUML, draw.io, Visual Paradigm gibi programlarla kolayca çizilebiliyor.

2. UML Diyagram Türleri

UML 2.5'te toplam 14 tane diyagram var ve bunlar iki gruba ayrılıyor.

a) Yapısal Diyagramlar: Sistemin nelerden oluştuğunu gösterir. Sistemin bir fotoğrafını çekmek gibi düşünülebilir, zamanla değişmez.

- **Class (Sınıf) Diyagramı:** Sınıfları, özelliklerini, metotlarını ve sınıfların birbiriyle ilişkisini gösterir. En çok kullanılan diyagram budur.
- **Object (Nesne) Diyagramı:** Belli bir andaki nesneleri ve değerlerini gösterir.

- **Component (Bileşen) Diyagramı:** Sistemin büyük parçalarını (modüller, kütüphaneler gibi) gösterir.
- **Deployment (Dağıtım) Diyagramı:** Yazılımın hangi sunucu ya da cihazda çalışacağını gösterir.
- **Package (Paket) Diyagramı:** Sınıfları klasör mantığıyla gruplar.
- **Composite Structure (Bileşik Yapı) Diyagramı:** Bir sınıfın iç yapısını gösterir.
- **Profile (Profil) Diyagramı:** UML'ye özel eklemeler yapmak için kullanılır.

b) Davranışsal Diyagramlar: Sistemin nasıl çalıştığını, ne yaptığını gösterir.

- **Use Case (Kullanım Senaryosu) Diyagramı:** Kullanıcıların sistemde neler yapabildiğini gösterir.
- **Activity (Aktivite) Diyagramı:** Bir işin adım adım nasıl yapıldığını gösterir, akış şemasına benzer.
- **State Machine (Durum) Diyagramı:** Bir nesnenin geçtiği durumları gösterir. Mesela bir sipariş: Hazırlanıyor, Kargoda, Teslim Edildi.
- **Sequence (Sıralama) Diyagramı:** Nesnelerin birbirine hangi sırayla mesaj gönderdiğini gösterir.
- **Communication (İletişim) Diyagramı:** Sequence'e benziyor ama nesneler arasındaki bağlantılara odaklanıyor.
- **Timing (Zamanlama) Diyagramı:** Olayların zamana göre nasıl değiştiğini gösterir.
- **Interaction Overview (Etkileşime Genel Bakış) Diyagramı:** Birkaç etkileşim diyagramını bir arada gösterir.

Burada şunu da belirtmem lazım: **ER diyagramı aslında UML'nin bir parçası değil.** ER diyagramını 1976'da Peter Chen bulmuş, yani UML'den daha eski. Ama veritabanı tasarlarken çok kullanıldığı ve Class diyagramına benzediği için genelde UML ile birlikte anlatılıyor.

Ödevdeki örneklerin hepsini aynı proje üzerinden yaptım ki birbiriyle bağlantılı olsun. Proje bir **Online Kütüphane Sistemi**. Sitede ziyaretçiler kitap arayabiliyor, üye olanlar kitap ödünç alıp iade edebiliyor ve rezervasyon yapabiliyor. Kütüphane görevlisi de kitap ekleyip üyeleri yönetiyor. Kitap geç getirilirse ceza çıkıyor ve üyeye mail gidiyor.

3. Use Case Diyagramı

Use case diyagramı sistemi kullanıcının gözünden anlatıyor. "Bu sistemi kimler kullanacak ve ne yapabilecekler?" sorusunun cevabı. Sistemin içinde işlerin nasıl yapıldığıyla ilgilenmiyor, sadece ne yapıldığını gösteriyor. Genelde projenin en başında çiziliyor ve çok basit olduğu için yazılımdan anlamayan biri bile okuyabiliyor.

Use case diyagramının elemanları:

- **Aktör:** Sistemi kullanan kişi ya da sistemdir. Çöp adam şeklinde çizilir. Burada önemli olan kişi değil rol. Ayrıca aktör insan olmak zorunda değil, mesela bir e-posta sistemi de aktör olabilir.
- **Use Case:** Aktörün sistemde yaptığı iştir. Oval (elips) şeklinde çizilir ve genelde "Kitap Ara", "Giriş Yap" gibi fiille adlandırılır.
- **Sistem Sınırı:** Use case'lerin içine çizildiği büyük dikdörtgendir. Aktörler bu kutunun dışında kalır.
- **İlişkiler:** Aktörlerle use case'leri ve use case'leri kendi aralarında bağlayan çizgilerdir.

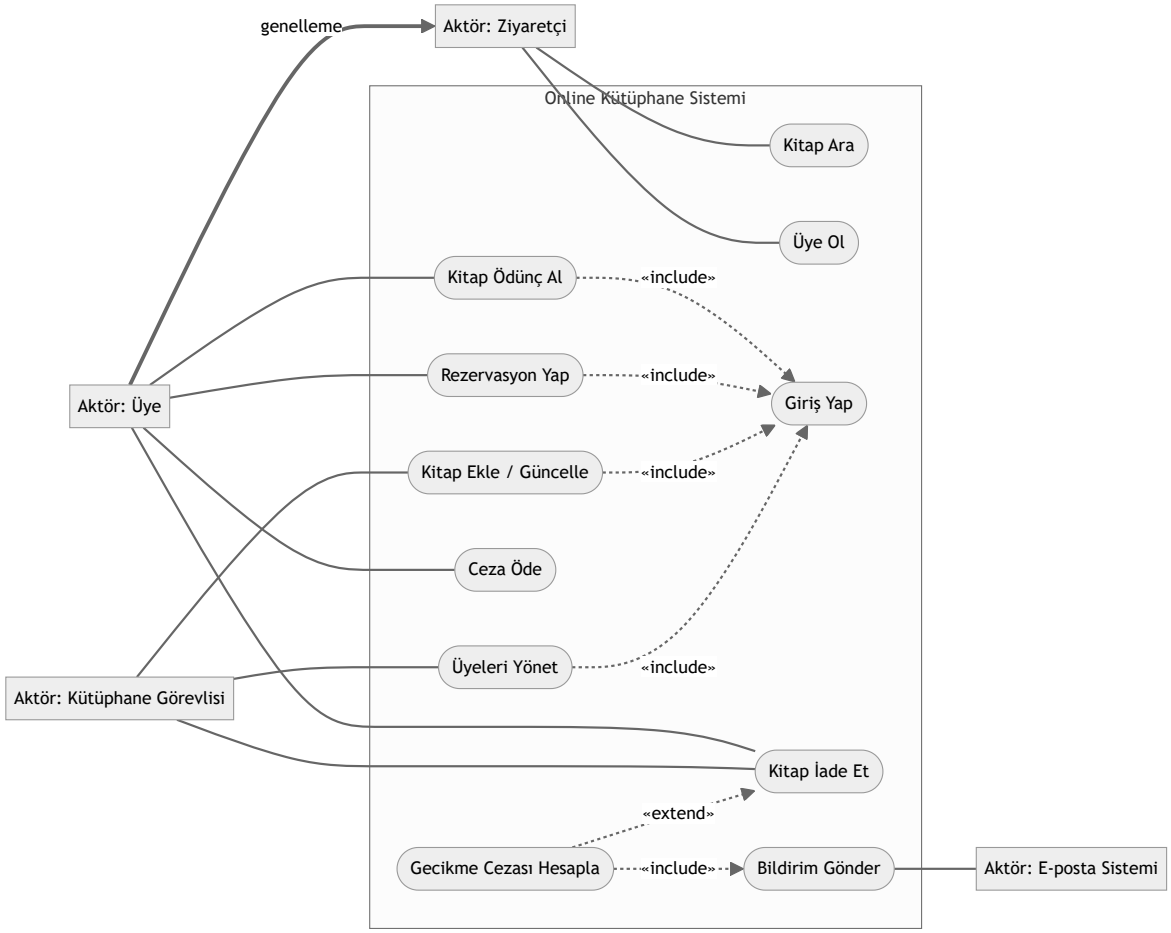
İlişki türleri:

1. **Association (İlişkilendirme):** Aktör ile use case arasındaki düz çizgi. "Bu aktör bu işi yapabilir" demek.
2. **Include (Dahil etme):** Bir iş yapılırken başka bir işin de **mutlaka** yapılması gerektiğini gösterir. Kesikli okla çizilir ve üzerine «include» yazılır. Örneğin kitap ödünç almak için her seferinde giriş yapmak zorunlu, bu yüzden "Kitap Ödünç Al" ile "Giriş Yap" arasında include var.
3. **Extend (Genişletme):** Bir işin **sadece bazı durumlarda** başka bir işi tetiklediğini gösterir. Bu da kesikli okla çizilir ama ok ters yöne, yani genişleten taraftan ana işe doğru gider. Örneğin her kitap iadesinde ceza çıkmaz, sadece kitap geç getirildiyse çıkar. Bu yüzden "Gecikme Cezası Hesapla" use case'i "Kitap İade Et" use case'ini extend ediyor.
4. **Generalization (Genelleme):** Kalıtım mantığı. Ucu boş üçgen olan okla çizilir. Örneğin "Üye" aslında bir "Ziyaretçi"dir. Ziyaretçinin yapabildiği her şeyi yapabilir, artı olarak kitap ödünç alma gibi işleri de yapabilir.

Include ile extend'i ilk başta ben de karıştırıyordum. Aklımda şöyle kaldı: include "her zaman", extend "bazen".

Örnek: Online Kütüphane Use Case Diyagramı

(Kullandığım araçta çöp adam ve elips şekli olmadığı için aktörleri köşeli kutuyla, use case'leri yuvarlak kutuyla çizdim.)



Diyagramı açıklayayım:

Sistemde 4 tane aktör var:

- **Ziyaretçi:** Siteye girip üye olmayan kişi. Sadece kitap arayabiliyor ve üye olabiliyor.
- **Üye:** Ziyaretçiden genelleme ile türediği için kitap arama gibi onun yaptığı her şeyi yapabiliyor. Bunlara ek olarak kitap ödünç alabiliyor, iade edebiliyor, rezervasyon yapabiliyor ve cezasını ödeyebiliyor.
- **Kütüphane Görevlisi:** Kitap ekleyip güncelliyor ve üyeleri yönetiyor. Kütüphaneye elden getirilen kitapların iadesini de sisteme o girdiği için "Kitap İade Et" ile de bağlantısı var.
- **E-posta Sistemi:** İnsan olmayan bir aktör. Ceza çıktığında üyeye giden maili bu sistem gönderiyor.

İlişkilere bakarsak; kitap ödünç alma, rezervasyon yapma, kitap ekleme ve üye yönetme işlerinin hepsi giriş yapmadan olmayacağı için hepsi include ile "Giriş Yap"a bağlı. "Gecikme Cezası Hesapla" ise sadece kitap geç getirilince çalıştığı için extend ile "Kitap İade Et"e bağlı. Ceza hesaplanınca da üyeye her zaman bildirim gittiği için "Gecikme Cezası Hesapla" ile "Bildirim Gönder" arasında include kullandım.

Kitap Ödünç Al senaryosu:

Use case diyagramının yanında önemli işler için genelde kısa bir senaryo da yazılıyor. Ben "Kitap Ödünç Al" için şöyle yazdım:

- **Aktör:** Üye
- **Ön koşul:** Üye giriş yapmış olmalı ve ödenmemiş cezası olmamalı.
- **Normal akış:**
    1. Üye kitabı arar ve kitabın sayfasını açar.
    2. "Ödünç Al" butonuna basar.
    3. Sistem kitabın rafta olup olmadığına bakar.
    4. Sistem üyenin elinde 3'ten fazla kitap olup olmadığına bakar.
    5. Sistem ödünç kaydını oluşturur ve iade tarihini 15 gün sonrası yapar.
    6. Ekranda "Kitap ödünç alındı" mesajı çıkar.
- **Farklı durumlar:** Kitap başkasındaysa sistem rezervasyon yapmak isteyip istemediğini sorar. Üyenin elinde zaten 3 kitap varsa uyarı verir ve işlem yapılmaz.
- **Son durum:** Kitabın durumu "Ödünçte" olur ve kitap üyenin listesine eklenir.

4. ER (Varlık-İlişki) Diyagramı

ER diyagramı, Entity-Relationship yani Varlık-İlişki diyagramı. Veritabanını tasarlamak için kullanılıyor. Veritabanında hangi tabloların olacağını, tablolarda hangi sütunların olacağını ve tabloların birbirine nasıl bağlanacağını gösteriyor. Web projelerinde neredeyse her zaman MySQL gibi bir veritabanı kullandığımız için bizim için önemli bir konu. Önce ER diyagramını çizip sonra tabloları oluşturunca "şu sütunu unutmuşum" gibi sorunlar azalıyor.

ER diyagramının elemanları:

Varlık (Entity): Hakkında bilgi tutmak istediğimiz şey. Veritabanında tabloya denk geliyor. Örneğin ÜYE, KİTAP, YAZAR. Dikdörtgenle çizilir. Bir de **zayıf varlık** diye bir şey var; bu da başka bir varlık olmadan var olamayan varlık. Örneğin bir siparişin detay satırları, sipariş silinirse bir anlamı kalmaz. Zayıf varlık çift çizgili dikdörtgenle çizilir.

Nitelik (Attribute): Varlığın özellikleri, yani tablonun sütunları. Elipsle çizilir. Birkaç türü var:

- **Anahtar nitelik (Primary Key):** Her kaydı diğerlerinden ayıran alan. Örneğin uye_id. Altı çizili gösterilir.
- **Basit nitelik:** Parçalara bölünemeyen alan. Örneğin ISBN numarası.

- **Birleşik nitelik:** Parçalara bölünebilen alan. Örneğin "ad soyad" alanı ad ve soyad diye ikiye bölünebilir.
- **Çok değerli nitelik:** Birden fazla değer alabilen alan. Örneğin bir üyenin birden fazla telefon numarası olabilir. Çift elipsle çizilir.
- **Türetilmiş nitelik:** Başka bir alandan hesaplanan alan. Örneğin yaş, doğum tarihinden hesaplanabilir. Kesikli elipsle çizilir.
- **Yabancı anahtar (Foreign Key):** Başka bir tablonun anahtarını tutan alan. Tabloları birbirine bu sayede bağlıyoruz.

İlişki (Relationship): Varlıkların arasındaki bağlantı. Baklava dilimi şeklinde çizilir ve genelde fiille adlandırılır. Örneğin "Üye kitabı ödünç alır".

Kardinalite: Bir tablodaki bir kaydın diğer tablodaki kaç kayıtla eşleşebileceğini gösteriyor. Bence ER diyagramının en önemli kısmı burası. Üç çeşit var:

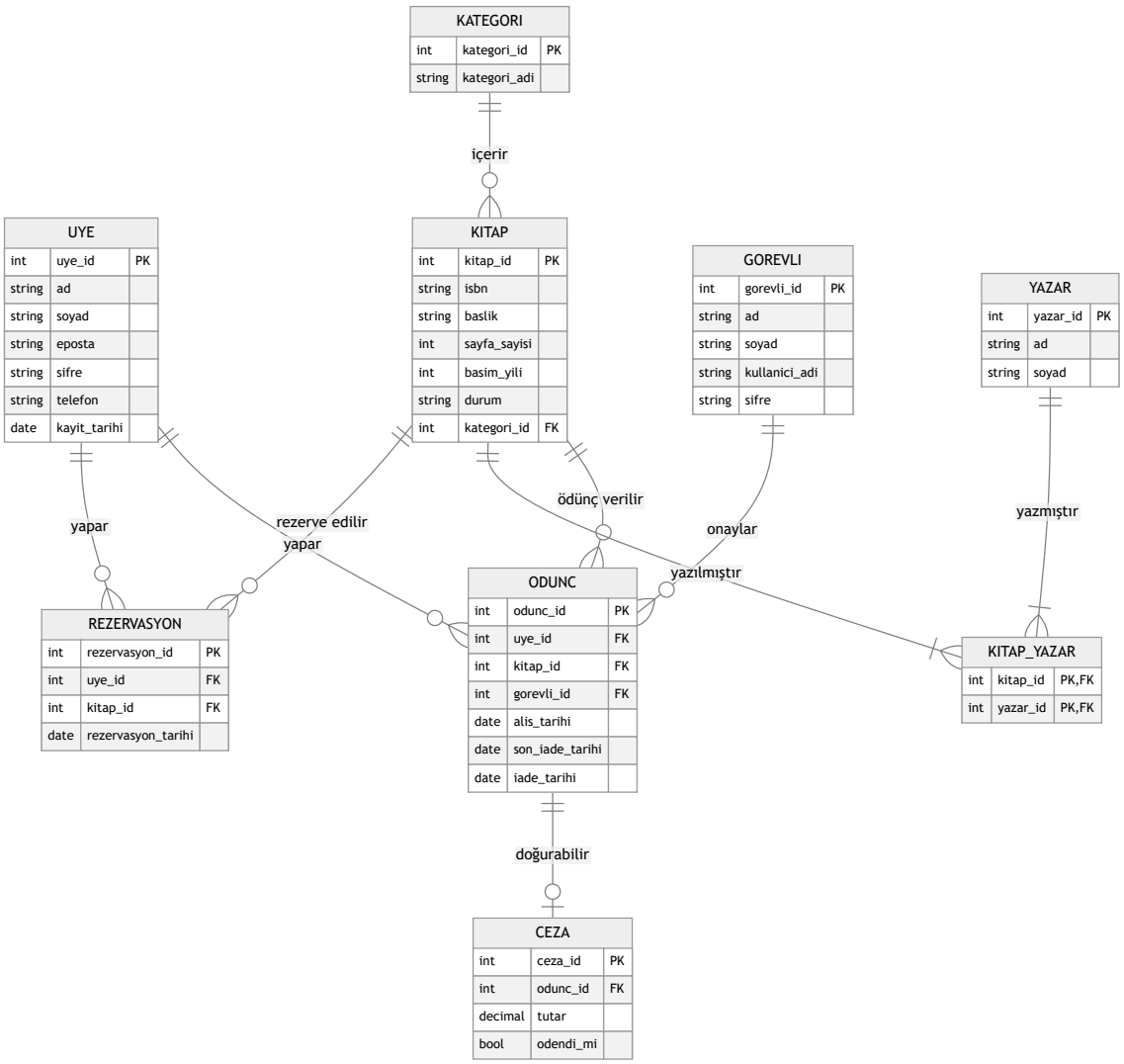
- **Bire-bir (1:1):** Bir kayıt karşı taraftaki sadece bir kayıtla eşleşir. Örneğin bir kişinin bir TC kimlik numarası olur, bir TC numarası da bir kişiye aittir.
- **Bire-çok (1:N):** Bir kayıt karşı taraftaki birden fazla kayıtla eşleşir. Örneğin bir kategoride birçok kitap olur ama bir kitap tek bir kategoriye aittir.
- **Çoka-çok (N:M):** İki taraf da birden fazla kayıtla eşleşebilir. Örneğin bir kitabın birden fazla yazarı, bir yazarın da birden fazla kitabı olabilir.

Çoka-çok ilişkide dikkat edilmesi gereken bir şey var: **veritabanında çoka-çok ilişki direkt kurulamıyor.** Bunun için araya bir **ara tablo** koyuluyor ve ilişki iki tane bire-çok ilişkiye bölünüyor. Benim örneğimde KITAP ile YAZAR arasındaki KITAP_YAZAR tablosu bunun için var.

Katılım: Bir de ilişkiye katılımın zorunlu ya da isteğe bağlı olması var. Örneğin her kitabın bir kategorisi olmak zorunda (zorunlu katılım), ama her üyenin kitap almış olması gerekmiyor (isteğe bağlı katılım).

Çizim şekilleri: ER diyagramı iki şekilde çizilebiliyor. Birincisi Chen gösterimi; derste gördüğümüz dikdörtgen, elips ve baklava dilimli şekil bu. İkincisi Crow's Foot (karga ayağı) gösterimi. Bunda nitelikler tablonun kutusunun içine yazılıyor ve çizgilerin ucundaki işaretler kardinaliteyi gösteriyor. Çatal şeklindeki uç "çok", tek düz çizgi "bir", daire de "sıfır olabilir" demek. MySQL Workbench gibi programlar bu gösterimi kullandığı için ben de örneği bununla çizdim.

Örnek: Online Kütüphane ER Diyagramı



(PK: birincil anahtar, FK: yabancı anahtar)

Diyagramı açıklayayım:

Sistemde 9 tane tablo var:

- **UYE:** Siteye kayıtlı üyeler.
- **GOREVLI:** Kütüphanede çalışanlar.
- **KATEGORI:** Roman, tarih, bilim gibi kategoriler.
- **KITAP:** Kütüphanedeki kitaplar. "durum" alanında kitabın rafta mı ödünçte mi olduğu tutuluyor.
- **YAZAR:** Kitapların yazarları.
- **KITAP_YAZAR:** Kitap ile yazar arasındaki çoka-çok ilişki için yaptığım ara tablo.

- **ODUNC:** Hangi üyenin hangi kitabı ne zaman aldığını ve ne zaman getirdiğini tutuyor.
- **CEZA:** Geç getirilen kitaplar için kesilen cezalar.
- **REZERVASYON:** Başkasında olan bir kitap için sıraya girme kayıtları.

Tablolar arasındaki ilişkiler de şöyle:

- **UYE - ODUNC (1'e çok):** Bir üye hiç kitap almamış olabilir ya da birçok kez kitap almış olabilir. Ama her ödünç kaydı tek bir üyeye ait.
- **KITAP - ODUNC (1'e çok):** Bir kitap zamanla birçok kez ödünç verilebilir. Her ödünç kaydında ise tek bir kitap var.
- **GOREVLI - ODUNC (1'e çok):** Her ödünç işlemini bir görevli onaylıyor, bir görevli de birçok işlemi onaylayabiliyor.
- **KATEGORI - KITAP (1'e çok):** Bir kategoride birçok kitap olabilir ama her kitap tek bir kategoride.
- **KITAP - YAZAR (çoka çok):** Bir kitabın birden fazla yazarı, bir yazarın birden fazla kitabı olabileceği için KITAP_YAZAR ara tablosu ile ikiye böldüm. Bu tablonun anahtarı kitap_id ve yazar_id'nin ikisinden birlikte oluşuyor.
- **ODUNC - CEZA (1'e 0 ya da 1):** Bir ödünç işleminden ya hiç ceza çıkmaz ya da en fazla bir ceza çıkar. Ceza da ödünç işlemi olmadan olamayacağı için zayıf varlık gibi düşünülebilir.
- **UYE ve KITAP - REZERVASYON (1'e çok):** Bir üye birçok rezervasyon yapabilir, bir kitap için de birden fazla kişi sıraya girebilir.

ER diyagramı bittikten sonra tabloları oluşturmak çok kolay oluyor. Örneğin ODUNC tablosunu MySQL'de şöyle oluşturabiliriz:

```
CREATE TABLE odunc (
    odunc_id INT AUTO_INCREMENT PRIMARY KEY,
    uye_id INT NOT NULL,
    kitap_id INT NOT NULL,
    gorevli_id INT NOT NULL,
    alis_tarihi DATE NOT NULL,
    son_iade_tarihi DATE NOT NULL,
    iade_tarihi DATE NULL,
    FOREIGN KEY (uye_id) REFERENCES uye(uye_id),
    FOREIGN KEY (kitap_id) REFERENCES kitap(kitap_id),
    FOREIGN KEY (gorevli_id) REFERENCES gorevli(gorevli_id)
);
```

Yani diyagramdaki her varlık bir tablo, her nitelik bir sütun oluyor. Bire-çok ilişkilerde de "çok" olan tarafa yabancı anahtar ekleniyor.

5. Class (Sınıf) Diyagramı

Class diyagramı nesne yönelimli programlamadaki sınıfları, bu sınıfların özelliklerini ve metotlarını, bir de sınıflar arasındaki ilişkileri gösteriyor. UML diyagramları içinde en çok kullanılanı bu, çünkü direkt koda dönüşüyor. Diyagramdaki her sınıf kodda bir class oluyor.

ER diyagramı ile karıştırılabiliyor ama arada önemli bir fark var. ER diyagramı veritabanını, yani sadece veriyi gösteriyor. Class diyagramı ise kodu gösteriyor ve veriyle birlikte metotları, yani sınıfın ne yapabildiğini de içeriyor.

Bir sınıf nasıl çizilir?

Sınıf üç bölmeli bir dikdörtgenle çiziliyor. En üstte sınıfın adı, ortada özellikleri (nitelikler), en altta da metotları yazıyor. Özellikler "görünürlük ad : tür" şeklinde, metotlar da "görünürlük ad(parametreler) : dönüş tipi" şeklinde yazılıyor. Örneğin `- baslik : string` ya da `+ oduncVerilebilirMi() : bool` gibi.

Soyut (abstract) sınıflar italik yazılıyor ya da üstüne «abstract» yazılıyor. Interface'ler için de «interface» yazılıyor. Statik olan özelliklerin ise altı çiziliyor.

Görünürlük işaretleri:

- **+ (public):** Her yerden erişilebilir.
- **- (private):** Sadece sınıfın kendi içinden erişilebilir.
- **# (protected):** Sınıfın kendisi ve ondan türeyen alt sınıflar erişebilir.
- **~ (package):** Aynı paketteki sınıflar erişebilir.

Genelde özellikler private, metotlar public yapılıyor. Böylece verilere dışarıdan direkt değil metotlar üzerinden erişiliyor. Buna kapsülleme (encapsulation) deniyor.

Sınıflar arasındaki ilişkiler:

Bu kısım en çok karıştırılan kısım. Zayıftan güçlüye doğru şöyle:

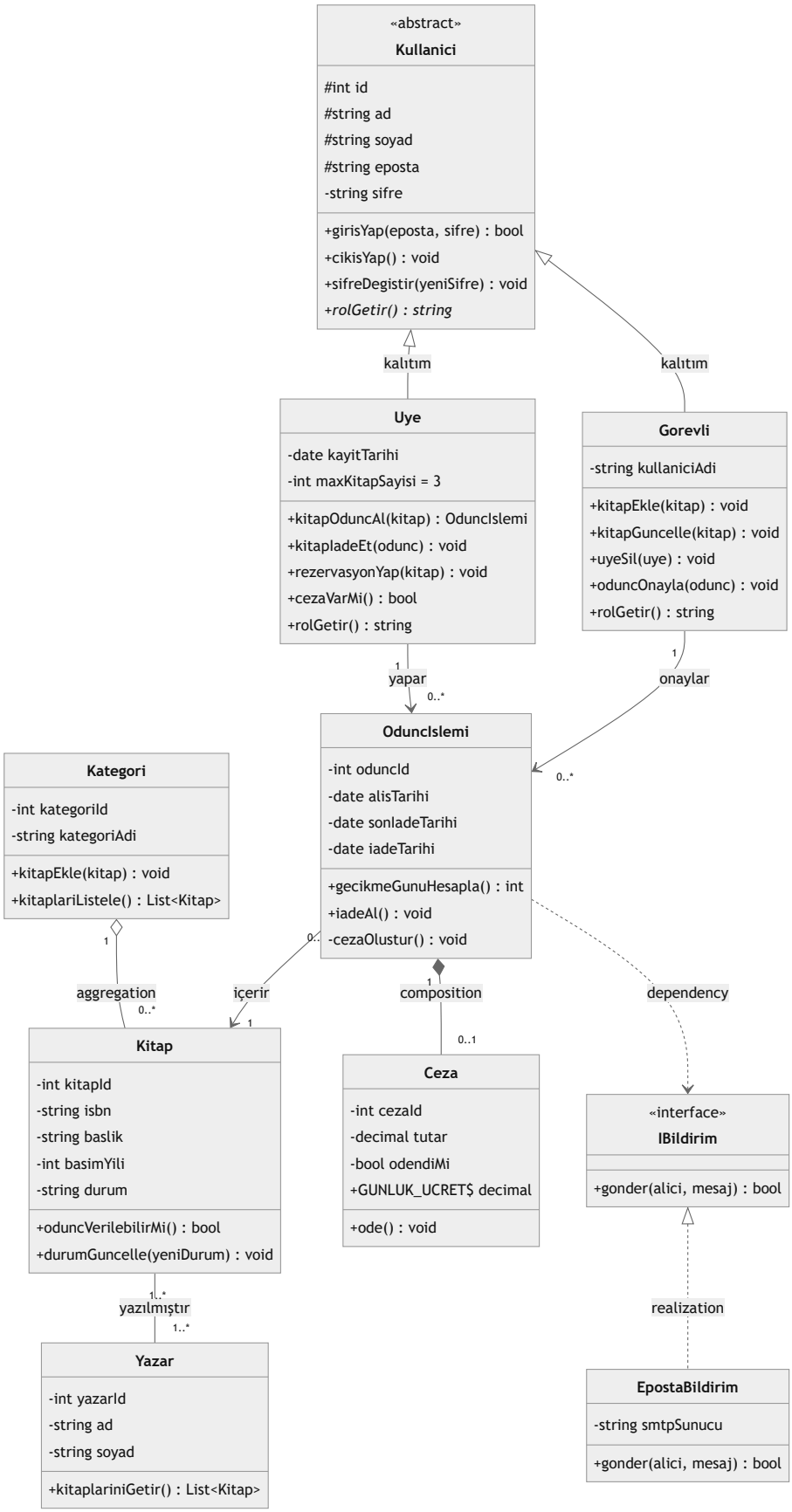
1. **Dependency (Bağımlılık):** Bir sınıf başka bir sınıfı sadece geçici olarak kullanıyor. Örneğin bir metotta parametre olarak alıyor. Kesikli çizgi ve okla çizilir. En zayıf ilişki bu.
2. **Association (İlişkilendirme):** İki sınıf birbirini tanıyor, aralarında kalıcı bir bağlantı var. Düz çizgiyle çizilir.
3. **Aggregation (Kümeleme):** "Sahip olma" ilişkisi ama parça, bütün olmadan da yaşayabiliyor. Bütünün tarafına içi boş elmas çizilir. Örneğin okul ve öğrenci; okul kapansa da öğrenciler var olmaya devam eder.

4. **Composition (Birleşim):** Daha güçlü bir sahip olma ilişkisi. Parça, bütün olmadan var olamıyor, bütün silinince parça da siliniyor. İçi dolu elmasla çizilir. Örneğin ev ve odalar; ev yıkılınca odalar da gider.
5. **Inheritance (Kalıtım):** Bir sınıfın başka bir sınıftan türemesi. "Bir türüdür" ilişkisi diye de geçiyor. Ucu boş üçgen olan düz okla çizilir.
6. **Realization (Gerçekleştirme):** Bir sınıfın bir interface'i uygulaması (implements). Ucu boş üçgen olan kesikli okla çizilir.

Aggregation ile composition'ı ayırmak için kendime "bütünü silersem parçası da silinmeli mi?" diye soruyorum. Cevap evetse composition, hayırsa aggregation.

Çokluk (Multiplicity): İlişki çizgilerinin uçlarına kaç nesnenin ilişkiye katıldığını gösteren sayılar yazılıyor. ER'deki kardinalitenin aynısı gibi. "1" tam olarak bir, "0..1" sıfır ya da bir, "*" ya da "0..*" sıfır ya da daha fazla, "1..*" en az bir demek.

Örnek: Online Kütüphane Class Diyagramı



Diyagramı açıklayayım:

Kullanici sınıfını abstract yaptım. Üye ve görevlinin ortak özelliklerini (id, ad, soyad, e-posta, şifre) ve ortak metotlarını (giriş yap, çıkış yap, şifre değiştir) bu sınıfa koydum. Abstract olduğu için bu sınıftan direkt nesne oluşturulamıyor. Özelliklerin çoğu protected (#) çünkü alt sınıfların bunlara erişmesi lazım. Ama şifreyi private (-) yaptım, çünkü şifreye alt sınıflar bile direkt erişmemeli. rolGetir() metodu da abstract, yani her alt sınıf bu metodu kendisi yazmak zorunda.

Uye ve **Gorevli** sınıfları Kullanici'dan kalıtım alıyor. Böylece ortak kodları iki kere yazmamış oluyorum. Üye sınıfında kitap ödünç alma, iade etme gibi metotlar var, görevli sınıfında da kitap ekleme ve üye silme gibi metotlar var.

Kitap, **Yazar** ve **Kategori** temel sınıflar. Kitap'taki oduncVerilebilirMi() metodu kitap raftaysa true döndürüyor.

OduncIslemi sistemin ana sınıfı diyebilirim. Gecikme gününü hesaplıyor ve iade işlemini yapıyor. cezaOlustur() metodunu private yaptım, çünkü dışarıdan çağrılmasını istemiyorum. Bu metot sadece iadeAl() içinde, kitap geç getirildiyse çalışıyor.

Ceza sınıfındaki GUNLUK_UCRET statik bir alan (diyagramda $ ile gösterilmiş). Yani bütün ceza nesneleri için aynı değer kullanılıyor.

IBildirim bir interface, **EpostaBildirim** de bu interface'i uyguluyor. Bunu böyle yapmamın sebebi, ileride SMS ile bildirim eklemek istersem sadece SmsBildirim diye yeni bir sınıf yazmamın yeterli olması. Diğer kodları değiştirmem gerekmiyor.

Sınıflar arasındaki ilişkiler:

- **Uye ve Gorevli, Kullanici'dan kalıtım alıyor.** Üye bir kullanıcıdır, görevli de bir kullanıcıdır.
- **Uye ile OduncIslemi arasında association var.** Bir üyenin hiç ya da birçok ödünç işlemi olabilir.
- **OduncIslemi ile Kitap arasında association var.** Her ödünç işleminde bir kitap var.
- **Gorevli ile OduncIslemi arasında association var.** Bir görevli birçok işlemi onaylayabilir.
- **OduncIslemi ile Ceza arasında composition var.** Ceza ödünç işleminin bir parçası, ödünç işlemi silinirse ceza da silinir.
- **Kategori ile Kitap arasında aggregation var.** Kategori silinse bile kitaplar yok olmaz, başka kategoriye taşınabilir.

- **Kitap ile Yazar arasında çoka-çok association var.** ER diyagramındaki gibi ara tabloya gerek yok, iki sınıf da birbirinin listesini tutabilir.
- **EpostaBildirim ile IBildirim arasında realization var.** EpostaBildirim, IBildirim'i implement ediyor.
- **OduncIslemi ile IBildirim arasında dependency var.** OduncIslemi, ceza çıkınca mail göndermek için IBildirim'i sadece o an kullanıyor.

Ders web programcılığı olduğu için diyagramın bir kısmının PHP'de nasıl yazılacağını da göstermek istedim:

```
<?php
abstract class Kullanici {
    protected $id;
    protected $ad;
    protected $eposta;
    private $sifre;

    public function girisYap($eposta, $sifre) {
        return $this->eposta == $eposta && password_verify($sifre, $this->sifre);
    }

    abstract public function rolGetir();
}

class Uye extends Kullanici {
    private $maxKitapSayisi = 3;

    public function rolGetir() {
        return "Üye";
    }
}

interface IBildirim {
    public function gonder($alici, $mesaj);
}

class EpostaBildirim implements IBildirim {
    public function gonder($alici, $mesaj) {
        // mail gönderme kodu
        return true;
    }
}
```

Burada abstract sınıfın, kalıtımın (extends) ve interface'in (implements) diyagramdaki karşılıkları görülüyor.

6. Sonuç

Bu ödevi yaparken üç diyagramın aslında aynı sistemi farklı açılardan gösterdiğini fark ettim. Use case diyagramı sistemin ne yapacağını, ER diyagramı verilerin veritabanında nasıl tutulacağını, class diyagramı da kodun nasıl olacağını gösteriyor. Mesela use case diyagramında "Kitap Ödünç Al" diye bir iş belirledim, ER diyagramında bunun için ODUNC tablosunu yaptım, class diyagramında da bu işi yapan OduncIslemi sınıfını oluşturdum.

Diyagramları çizerken projeyle ilgili kod yazarken aklıma gelmeyecek birçok şeyi düşünmem gerekti. Mesela giriş yapmadan rezervasyon yapılıp yapılamayacağını, kitap ile yazar arasında ara tablo gerektiğini ya da üye ile görevlinin ortak özelliklerini tek bir üst sınıfta toplayabileceğimi bu sayede fark ettim. Bunları kod yazarken fark etseydim birçok yeri baştan yazmam gerekirdi. Bu yüzden UML çizmek başta fazladan iş gibi görünse de sonradan zaman kazandırıyor.

Kaynaklar

- OMG, UML 2.5.1 Spesifikasyonu, https://www.omg.org/spec/UML/
- Martin Fowler, UML Distilled, Addison-Wesley
- Peter Chen, "The Entity-Relationship Model", 1976
- Visual Paradigm UML rehberi, https://www.visual-paradigm.com/guide/uml-unified-modeling-language/
- Web Programcılığı III ders notları